

PYTHON MINI PROJECT REPORT

Project Title	Hydrostatic Force & Centre of Pressure on Submerged Plates	
Course Name	Semester-3: Python Programming	
Class Name	Mechanical-3A	
Branch Name	Mechanical Engineering / Automation & Robotics	
Prepared By	Lalluvadiya Jenil Sagarbhai	25012251210001
Guided By	Mr. MohammedAzim Shaikh	
Submitted to	Department of Mechanical Engineering, LJ Polytechnic, Lok Jagruti Kendra University, Ahmedabad, Gujarat, India.	

CERTIFICATE

This certificate is proudly presented to the following student for successfully completing the Python Mini-Project entitled “Hydrostatic Force & Centre of Pressure on Submerged Plates” as part of the Diploma in Automation & Robotics Engineering curriculum. This project work has been carried out under my guidance and supervision and it is up to my satisfaction.

Name of Students	Enrolment Number	Class
Lalluvadiya Jenil Sagarbhai	25012251210001	Mechanical-3A

Date: 25 September 2026	Name & Signature of Guide
Place: Ahmedabad	Mr. MohammedAzim Shaikh

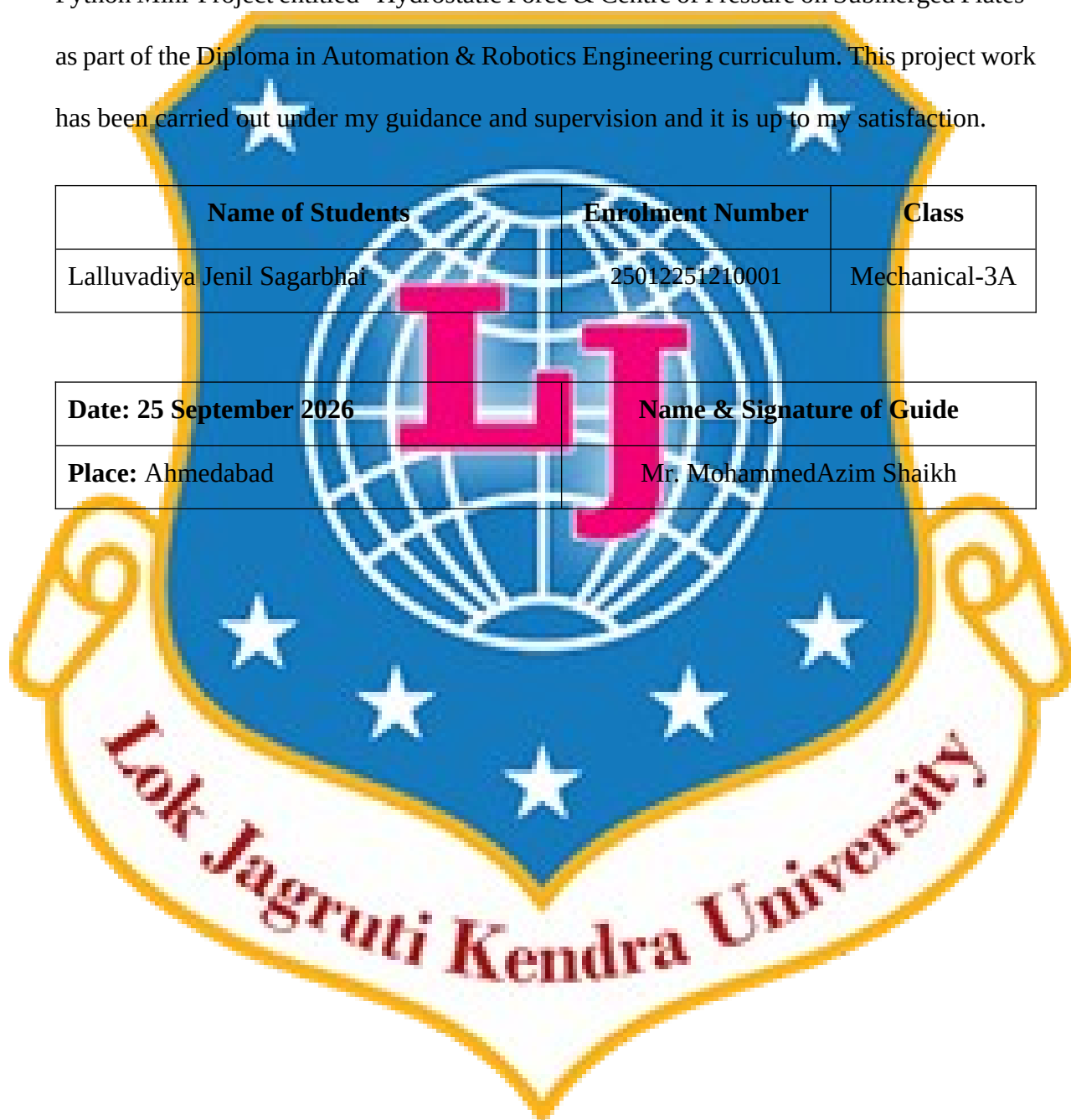


TABLE OF CONTENTS

CERTIFICATE.....	2
TABLE OF CONTENTS.....	3
1. INTRODUCTION.....	4
2. OBJECTIVE.....	4
3. METHODOLOGY.....	4
4. MECHANICAL ENGINEERING APPLICATION.....	5
5. PYTHON CODE.....	5
6. SCREENSHOTS.....	7
7. RESULTS AND DISCUSSIONS.....	9
8. CONCLUSION.....	10
9. REFERENCES.....	10
10. COURSE CERTIFICATES.....	11

1. INTRODUCTION

This mini-project builds an interactive Python (Streamlit) calculator that finds the total hydrostatic thrust and the centre of pressure on a submerged rectangular or circular plate. In fluid mechanics, gauge pressure increases linearly with depth. For a plate of finite size the pressure is therefore not uniform, so the resultant force does not act at the centroid. Designers of lock gates, dams, tanks, manholes and submarine hatches must know both the magnitude of the thrust and the point where it acts. Hand calculation of every new geometry is slow and easy to mis-type. The app accepts plate size, inclination, immersion depth, fluid density and gravity, then reports F , $\bar{h}$, h_{cp} and draws a live schematic with a pressure graph. The work connects a standard mechanical-engineering topic (hydrostatics of plane surfaces) with reusable Python code, input validation and visual checking.

The program is a solo project by Lalluvadiya Jenil Sagarbhai (Enrollment No. 25012251210001), Diploma in Automation & Robotics Engineering, Lok Jagruti Kendra University, prepared under the guidance of Mr. MohammedAzim Shaikh for Semester-3 Python Programming.

2. OBJECTIVE

The objective is to write a single Python program that (i) computes hydrostatic force and centre of pressure for rectangular and circular plates, vertical or inclined; (ii) supports several liquids plus a custom density; (iii) rejects impossible inputs with clear messages; (iv) shows an animated tank schematic (centroid C , centre of pressure CP , pressure arrows and resultant F); and (v) prints a step-by-step working that can be compared with a paper solution. The target user is a diploma student or junior engineer who wants an instant, checked numerical instead of repeating the same algebra.

3. METHODOLOGY

The workflow was: solve a textbook numerical on paper to obtain test values; code the same formulas in a `compute()` function; wrap the function in a Streamlit sidebar (shape, units, fluid, angle, animation); add HTML/JavaScript canvas animation and a Matplotlib pressure plot; validate against the paper answer; push the files to GitHub and deploy on Streamlit

Community Cloud. Libraries used are streamlit (widgets and layout), numpy and matplotlib (pressure graph), math (area and second moment), and json (passing results into the animation). All lengths are converted to metres internally using a unit dictionary (mm, cm, m, km, in, ft). Angle θ is taken from the free surface (90° = vertical). Validation blocks zero/negative sizes, negative depth, non-finite numbers, and a circular plate that would pierce the free surface.

Tools and files: Python 3, Streamlit, NumPy, Matplotlib, app.py, requirements.txt, GitHub repository, Streamlit Community Cloud. Animation speed and a Replay button let the examiner watch the tank fill, the plate drop into place, pressure arrows grow, and the red resultant F appear at the centre of pressure.

4. MECHANICAL ENGINEERING APPLICATION

- The project applies the plane-surface hydrostatics chapter of fluid mechanics, which is taught in mechanical, civil and automation & robotics diplomas. Gauge pressure at a point is $p = \rho g h$. Integrating p over the area gives the resultant thrust $F = \rho g \bar{h} A$, where $\bar{h}$ is the vertical depth of the centroid. The centre of pressure lies below the centroid because pressure is larger on the deeper part of the plate. The shift along the plate is $e = I_G / (\bar{y} A)$, so the vertical CP depth is $h_{cp} = \bar{h} + I_G \sin^2\theta / (A \bar{h})$. These results are used when sizing hinges, bolts and frames of sluice gates, tanker walls, inspection covers and submarine hatches. Changing fluid density (water $\rightarrow$ sea water $\rightarrow$ mercury) scales the force; tilting the plate reduces the vertical CP gap by $\sin^2\theta$. The submarine-hatch example in the app (circular lid, $R = 0.5$ m, centre 100 m deep in sea water) produces about 790 kN of thrust — roughly 80 tonnes — which shows why the calculation matters in machine and vehicle design.
- Example mapped to this work: the project uses Python to compute hydrostatic thrust and centre of pressure on lock gates, tank walls and hatches, and to plot how pressure grows with depth.

5. PYTHON CODE

The complete source code of the Streamlit application (file name: app.py) is printed below, exactly as submitted. It includes the fluid and unit tables, the compute() formulas, sidebar

inputs, error checks, metric cards, the HTML/JavaScript tank animation, the Matplotlib pressure graph, the step-by-step working expander, the paper-vs-app check, and the symbols table.

To run this program on a PC: install Python 3, then run `pip install streamlit numpy matplotlib` and start the app with `streamlit run app.py`

Listing: app.py — Hydrostatic Force & Centre of Pressure on Submerged Plates

```
"""
Hydrostatic Force & Centre of Pressure on Submerged Plates (Streamlit app)
Author      : Lalluvadiya Jenil Sagarbhai
Enrollment : 25012251210001
Program     : Diploma in Automation & Robotics Engineering, Lok Jagruti Kendra University
"""
import json
import math

import matplotlib.pyplot as plt
import numpy as np
import streamlit as st
import streamlit.components.v1 as components

st.set_page_config(page_title="Hydrostatic Force & Centre of Pressure", page_icon="🌊",
layout="wide")

FLUIDS = { # name: (density kg/m3, colour)
    "Fresh water": (1000, "#3fa7ff"),
    "Sea water": (1025, "#1d86b8"),
    "Oil (yellowish)": (900, "#e8b923"),
    "Kerosene": (810, "#f2d86b"),
    "Glycerin": (1260, "#bfe3c8"),
    "Mercury": (13600, "#aab4be"),
    "Custom fluid": (None, None),
}
UNITS = {"mm": 1e-3, "cm": 1e-2, "m": 1.0, "km": 1e3, "in": 0.0254, "ft": 0.3048}

# ----- helpers
def fnum(x, sig=5, tex=False):
    if x == 0:
        return "0"
    a = abs(x)
    if a >= 1e9 or a < 1e-3:
        m, e = f"{x:.{sig-1}e}".split("e")
        return f"{m}\\times10^{{{int(e)}}}" if tex else f"{x:.{sig-1}e}"
    s = f"{x:.{max(0, sig - 1 - int(math.floor(math.log10(a))))}f}"
    if "." in s:
        s = s.rstrip("0").rstrip(".")
    return s.replace(",", "{,}") if tex else s

def force_txt(F):
    for lim, u in ((1e9, "GN"), (1e6, "MN"), (1e3, "kN")):
        if F >= lim:
            return f"{fnum(F / lim)} {u}"
    return f"{fnum(F)} N"

def compute(shape, dim1, dim2, theta, depth, rho, g):
    """All lengths in metres. rect: dim1=b, dim2=h | circle: dim1=R.
    depth = H (top edge for rectangle, centre for circle), measured vertically."""
    s = math.sin(math.radians(theta))
    if shape == "Rectangular":
        A, IG, L = dim1 * dim2, dim1 * dim2 ** 3 / 12, dim2
        dT = depth
        hc = dT + L / 2 * s
    else:
        A, IG, L = math.pi * dim1 ** 2, math.pi * dim1 ** 4 / 4, 2 * dim1
        hc = depth
```

```

    dT = hc - dim1 * s
    ybar = hc / s                                # slant distance of centroid from free surface
    e = IG / (ybar * A)                          # CP lies e below centroid (along the plate)
    hcp = (ybar + e) * s
    F = rho * g * hc * A
    dB = dT + L * s
    return dict(A=A, IG=IG, L=L, dT=dT, dB=dB, hc=hc, hcp=hcp, e=e, F=F,
               fcp=(L / 2 + e) / L, pc=rho * g * hc, ybar=ybar)

# ----- header
st.markdown("""
<style>
.hero{background:#0b2a43;color:#eaf4fb;padding:18px 24px 26px;border-radius:10px;margin-
bottom:14px;
    background-image:url("data:image/svg+xml;utf8,<svg xmlns='http://www.w3.org/2000/svg'
width='120' height='10' viewBox='0 0 120 10'><path d='M0 5 Q15 0 30 5 T60 5 T90 5 T120 5'
fill='none' stroke='%233fa7ff' stroke-width='2' /></svg>");
    background-repeat:repeat-x;background-position:bottom left}
.hero h1{margin:0 0 6px;font-size:1.7rem;color:#fff}
.hero p{margin:2px 0;font-size:.92rem;color:#c9dfe5}
</style>
<div class="hero"><h1>Hydrostatic Force & Centre of Pressure on Submerged Plates</h1>
<p><b>Group:</b> Individual (solo project) & <b>Name:</b> Lalluvadiya Jenil
Sagarbhai & <b>Enrollment No.:</b> 25012251210001</p>
<p>Diploma in Automation & Robotics Engineering & <b>Lok Jagruti Kendra
University</p></div>
""", unsafe_allow_html=True)

# ----- sidebar inputs
sb = st.sidebar
sb.header("Inputs")
shape = sb.radio("Plate shape", ["Rectangular", "Circular"], horizontal=True)
unit = sb.selectbox("Length unit", list(UNITS), index=2)
uf = UNITS[unit]
sb.caption(f"All lengths below are in **{unit}**. No upper limit on any value.")
if shape == "Rectangular":
    b = sb.number_input("Width b", value=2.0, step=0.1, format="%g")
    h = sb.number_input("Height h (along the plate)", value=3.0, step=0.1, format="%g")
    H = sb.number_input("Depth H of top edge (vertical)", value=1.5, step=0.1, format="%g")
else:
    R = sb.number_input("Radius R", value=1.0, step=0.1, format="%g")
    H = sb.number_input("Depth H of centre (vertical)", value=3.0, step=0.1, format="%g")
theta = sb.slider("Plate angle θ with free surface (90° = vertical)", 1, 90, 90)

sb.header("Fluid")
fluid = sb.selectbox("Select fluid", list(FLUIDS))
if fluid == "Custom fluid":
    name = sb.text_input("Fluid name", "My fluid")
    rho = sb.number_input("Density ρ (kg/m³)", value=1000.0, step=10.0, format="%g")
    col = sb.color_picker("Fluid colour", "#3fa7ff")
else:
    (rho, col), name = FLUIDS[fluid], fluid
    sb.caption(f"ρ = {rho} kg/m³")
g = sb.number_input("Gravity g (m/s²)", value=9.81, step=0.01, format="%g")

sb.header("Animation")
play = sb.checkbox("Play animation", True)
speed = sb.select_slider("Speed", [0.5, 1.0, 2.0], value=1.0)

# ----- validation
if shape == "Rectangular":
    checks = [("Width b", b), ("Height h", h)]
else:
    checks = [("Radius R", R)]
checks += [("Density ρ", rho), ("Gravity g", g)]
errs = [f"**{n}** must be a positive, finite number." for n, v in checks if not
(math.isfinite(v) and v > 0)]
if not math.isfinite(H) or H < 0:
    errs.append("**Depth H** cannot be negative (the plate must be below the free surface).")
if not errs and shape == "Circular" and H * uf < R * uf * math.sin(math.radians(theta)) * (1 -
1e-12):
    errs.append(f"The circle would stick out of the liquid. Centre depth H must be at least "
f"R·sinθ = {fnum(R * math.sin(math.radians(theta)))} {unit}. Increase H or
reduce R / θ.")
if errs:
    for m in errs:
        st.error(m)

```

```

    st.stop()

if shape == "Rectangular":
    r = compute(shape, b * uf, h * uf, theta, H * uf, rho, g)
else:
    r = compute(shape, R * uf, 0, theta, H * uf, rho, g)
if not math.isfinite(r["F"]):
    st.error("These numbers are too large to compute. Reduce the values.")
    st.stop()
if (r["hcp"] - r["hc"]) / r["hc"] < 1e-4:
    st.info("The plate is very deep compared with its size, so the centre of pressure almost
    coincides with the centroid.")

# ----- results
c1, c2, c3 = st.columns(3)
c1.metric("Total hydrostatic thrust F", force_txt(r["F"]), help=f"= {fnum(r['F'])} N")
c2.metric("Centroid depth  $\bar{h}$ ", f"{fnum(r['hc'] / uf)} {unit}")
c3.metric("Centre of pressure depth  $h_{cp}$ ", f"{fnum(r['hcp'] / uf)} {unit}")
c4, c5, c6 = st.columns(3)
c4.metric("CP below centroid (along plate)", f"{fnum(r['e'] / uf)} {unit}")
c5.metric("Plate area A", f"{fnum(r['A'] / uf ** 2)} {unit}^2")
c6.metric("Gauge pressure at centroid", f"{fnum(r['pc'] / 1e3)} kPa")

# ----- animated schematic
HTML = """
<style>
html,body{margin:0;font-family:"Segoe UI",Arial,sans-serif}
.card{background:#fff;border:1px solid #cfdde8;border-radius:10px;overflow:hidden}
.bar{display:flex;gap:16px;align-items:center;flex-wrap:wrap;padding:8px 12px;font-size:13px;color:#0f2a3d;border-bottom:1px solid #e3ecf3}
button{background:#0b2a43;color:#fff;border:0;border-radius:6px;padding:5px 12px;cursor:pointer;font-size:13px}
.d{display:inline-block;width:11px;height:11px;border-radius:50%;margin-right:5px;vertical-align:-1px;border:2px solid #fff;box-shadow:0 0 1px #888}
canvas{display:block;width:100%;height:500px}
</style>
<div class="card"><div class="bar"><button id="re">Replay animation</button>
<span><i class="d" style="background:#111"></i>Centroid (C)</span>
<span><i class="d" style="background:#e0202a"></i>Centre of pressure (CP)</span>
<span>White arrows = pressure (longer with depth)</span></div><canvas id="c"></canvas></div>
<script>
const D=__DATA__;
const cv=document.getElementById('c'),c=cv.getContext('2d');
let W,H=500,dpr=window.devicePixelRatio||1,t0=performance.now();
function rs(){W=cv.getBoundingClientRect().width;cv.width=W*dpr;cv.height=H*dpr;}
addEventListener('resize',rs);rs();
document.getElementById('re').onclick=()=>{t0=performance.now();};
const ease=x=>{x=Math.min(1,Math.max(0,x));return x<.5?4*x*x*x:1-Math.pow(-2*x+2,3)/2};
const sg=(t,a,b)>=>ease((t-a)/(b-a));
const fmt=v=>{v=v/D.uf;if(v===0)return'0';const a=Math.abs(v);return(a>=1e6||a<1e-3)?v.toExponential(2):String(parseFloat(v.toPrecision(4)))};
const hx=h=>[1,3,5].map(i=>parseInt(h.substr(i,2),16));
const mix=(h,k,to)>=>{const a=hx(h),b=to==='w'?[255,255,255]:[8,24,40];return'rgb('+a.map((v,i)>=>Math.round(v+(b[i]-v)*k)).join(',')+')'};
const FONT='Segoe UI, Arial, sans-serif';
const
th=D.theta*Math.PI/180,sn=Math.sin(th),cs=Math.cos(th),L=D.L,dT=D.dTop,ext=L*sn,dB=dT+ext;
const span=Math.max(ext,L*.4),zoom=ext<.25*dB;
let wT,wB;
if(zoom){wT=dT-.6*span;wB=dB+.6*span;}else{wB=dB*1.2;wT=0;}
if(wT<=0)wT=-.06*wB;

function arrow(a,b,col,w,hd){
    const len=Math.hypot(b[0]-a[0],b[1]-a[1]);if(len<2)return;
    const an=Math.atan2(b[1]-a[1],b[0]-a[0]),h=Math.min(hd,len*.6);

c.shadowColor='rgba(5,20,35,.6)';c.shadowBlur=4;c.strokeStyle=col;c.fillStyle=col;c.lineWidth=w;
c.lineCap='round';
c.beginPath();c.moveTo(a[0],a[1]);c.lineTo(b[0]-Math.cos(an)*h*.8,b[1]-Math.sin(an)*h*.8);c.stroke();
c.beginPath();c.moveTo(b[0],b[1]);c.lineTo(b[0]-h*Math.cos(an-.4),b[1]-h*Math.sin(an-.4));
c.lineTo(b[0]-h*Math.cos(an+.4),b[1]-h*Math.sin(an+.4));c.closePath();c.fill();c.shadowBlur=0;
}
function dot(q,col,r){
c.beginPath();c.arc(q[0],q[1],r+2.5,0,7);c.fillStyle='#fff';c.fill();
c.beginPath();c.arc(q[0],q[1],r,0,7);c.fillStyle=col;c.fill();
}

```

```

function frame(now){
  const el=(now-t0)/1000*D.speed,T=D.animate?Math.min(1,el/4.2):1,tm=now/1000;
  c.setTransform(dpr,0,0,dpr,0,0);c.clearRect(0,0,W,H);
  const R0=W-560?W*.3:Math.min(215,W*.26),tL=60,tR=W-R0,yA=28,yB=H-18,tw=tR-tL,xh=L*cs;
  let sc=(yB-yA)/(wB-wT);if(xh>0)sc=Math.min(sc,tw*.62/xh);
  const Y=d=>yA+(d-wT)*sc;
  c.fillStyle='#f3f8fc';c.fillRect(0,0,W,H);
  // fluid
  const fill=sg(T,0,.22),ys=Y(0),y0=Math.max(ys,yA-6),lvl=yB-(yB-y0)*fill,amp=ys>yA?3:0;
  const
  gr=c.createLinearGradient(0,y0,0,yB);gr.addColorStop(0,mix(D.color,.12,'w'));gr.addColorStop(1,
  mix(D.color,.5,'k'));
  c.fillStyle=gr;c.globalAlpha=.92;c.beginPath();c.moveTo(tL,yB);
  for(let x=tL;x<=tR;x+=6)c.lineTo(x,lvl+amp*Math.sin(x*.03+tm*2)+amp*.5*Math.sin(x*.08-
  tm*1.3));
  c.lineTo(tR,yB);c.closePath();c.fill();c.globalAlpha=1;
  // floating bubbles in the liquid
  if(D.bub&&fill>.20)for(let i=0;i<22;i++){
    const phase=(tm*.07*D.speed+i*.137)%1;
    const x=tL+((i*97)%100)/100*tw;
    const y=yB-phase*(yB-lvl);
    const rr=1.5+(i%4)*0.8;
    c.beginPath();
    c.arc(x,y,rr,0,7);
    c.strokeStyle='rgba(255,255,255,.70)';
    c.lineWidth=1.2;c.stroke();
  }
  // tank + depth ruler
  c.strokeStyle='#5d7385';c.lineWidth=4;c.lineCap='butt';c.beginPath();c.moveTo(tL,yA-
  6);c.lineTo(tL,yB);c.lineTo(tR,yB);c.lineTo(tR,yA-6);c.stroke();
  const raw=(wB-Math.max(wT,0))/6,p=Math.pow(10,Math.floor(Math.log10(raw))),f=raw/p,st=(f<1.5?
  1:f<3.5?2:f<7.5?5:10)*p;
  c.font='11px '+FONT;c.textAlign='right';c.lineWidth=1;
  for(let d=Math.ceil(Math.max(wT,0)/st)*st,n=0;d<=wB&&n<30;d+=st,n++){
    const y=Y(d);if(y>yB-4)break;
    c.strokeStyle='#4b6072';c.beginPath();c.moveTo(tL-7,y);c.lineTo(tL,y);c.stroke();
    if(y>lvl){c.strokeStyle='rgba(255,255,255,.28)';c.beginPath();c.moveTo(tL+2,y);c.lineTo(tR-
    2,y);c.stroke();}
    c.fillStyle='#4b6072';c.fillText(fmt(d*D.uf),tL-10,y+4);}
  c.textAlign='left';c.fillStyle='#4b6072';c.fillText('Depth ('+D.unit+')',6,16);
  if(ys>yA){c.fillStyle='#0b2a43';c.font='bold 12px '+FONT;c.fillText('Free surface
  (0)',tR+20,ys+4);}
  else{c.font='11px '+FONT;c.fillText('free surface is above this view',tL+8,yA+12);}
  c.font='bold 13px '+FONT;c.lineWidth=3;c.strokeStyle='rgba(0,0,0,.45)';
  const ft=D.name+' (ρ = '+D.rho+' kg/m³)';c.strokeText(ft,tL+10,yB-
  10);c.fillStyle='ffff';c.fillText(ft,tL+10,yB-10);
  // plate
  const cx=tL+tw*.5,hw=xh*sc/2,xt=cx+hw,xb=cx-hw,yt=Y(dT),yb=Y(dB);
  const P=s=>[xt+(xb-xt)*s,yt+(yb-yt)*s];
  const drop=sg(T,.18,.42);
  c.save();c.translate(0,-(1-drop)*(yt+90));c.lineCap='butt';
  c.strokeStyle='#39454f';c.lineWidth=12;c.beginPath();c.moveTo(xt,yt);c.lineTo(xb,yb);c.stroke()
  ;
  c.strokeStyle='#a9b6c1';c.lineWidth=8;c.stroke();c.restore();
  // pressure arrows (normal to plate, pointing into it)
  const ar=sg(T,.42,.66),mx=Math.min(tw*.32,130);
  if(ar>0){
    const N=Math.max(3,Math.min(11,Math.round(Math.hypot(xt-xb,yt-yb)/34))),tails=[];
    for(let i=0;i<N;i++){
      const s=(i+.5)/N,q=P(s),len=mx*(dT+s*ext)/dB*ar;
      const tip=[q[0]-sn*8,q[1]-cs*8],tail=[q[0]-sn*(8+len),q[1]-cs*(8+len)];
      arrow(tail,tip,'#ffffff',2.4,9);tails.push(tail);}
    c.setLineDash([4,4]);c.strokeStyle='rgba(255,255,255,.9)';c.lineWidth=1.2;c.beginPath();
    tails.forEach((q,i)>=>i?
    c.lineTo(q[0],q[1]):c.moveTo(q[0],q[1]));c.stroke();c.setLineDash([]);}
  // depth guide lines
  const gl=sg(T,.72,.95);
  if(gl>0){
    c.globalAlpha=gl;
    const yc=Y(D.hc),yp=Y(D.hcp),lp=Math.max(yp,yc+34);
    const guide=(y,ly,col,ll,l2)>=>{
      c.setLineDash([6,4]);c.strokeStyle=col;c.lineWidth=1.4;c.beginPath();c.moveTo(tL,y);c.lineTo(tR
      +10,y);c.stroke();c.setLineDash([]);
      if(ly!==y){c.beginPath();c.moveTo(tR+10,y);c.lineTo(tR+18,ly-4);c.stroke();}
      c.textAlign='left';c.fillStyle=col;c.font='bold 13px '+FONT;c.fillText(l1,tR+20,ly);
      c.fillStyle='#4b6072';c.font='11px '+FONT;c.fillText(l2,tR+20,ly+14);}
  }
}

```

```

        guide(yc,yc,'#111','h̄ = '+fmt(D.hc)+' '+D.unit,'centroid depth');
        guide(yp,lp,'#e0202a','h_cp = '+fmt(D.hcp)+' '+D.unit,'centre of pressure depth');
        c.globalAlpha=1;
        // centroid, CP, resultant force
        const cg=sg(T,.6,.7),sl=sg(T,.7,.92),fo=sg(T,.85,1);
        if(cg>0)dot(P(.5),'#111',6*cg);
        if(cg>0){const q=P(.5+(D.fcp-.5)*sl);

c.beginPath();c.arc(q[0],q[1],9+3*Math.sin(tm*4),0,7);c.strokeStyle='rgba(224,32,42,.5)';c.lineWidth=2;c.stroke();
        dot(q,'#e0202a',6.5*cg);}
        if(fo>0){
            const q=P(D.fcp),Lf=(mx+50)*fo,tip=[q[0]-sn*10,q[1]-cs*10],tail=[q[0]-sn*(10+Lf),q[1]-
cs*(10+Lf)];
            arrow(tail,tip,'#e0202a',5,16);
            if(fo>.8){c.font='bold 14px
'+FONT;c.fillStyle='#e0202a';c.textAlign=sn>.5?'right':'center';
            c.lineWidth=3;c.strokeStyle='rgba(255,255,255,.85)';const tx=Math.max(tail[0]-sn*8,sn>.5?
100:60);ty=tail[1]-cs*12+4;
            c.strokeText('F = '+D.ftxt,tx,ty);c.fillText('F = '+D.ftxt,tx,ty);}
            requestAnimationFrame(frame);
        }
        requestAnimationFrame(frame);
</script>
"""
data = dict(L=r["L"], theta=theta, dTop=r["dT"], hc=r["hc"], hcp=r["hcp"], fcp=r["fcp"],
color=col,
            name=name, rho=fnum(rho), ftxt=force_txt(r["F"]), uf=uf, unit=unit,
            bub=rho < 5000, animate=play, speed=speed)
payload = json.dumps(data).replace("</", "<\\/")
components.html(HTML.replace("__DATA__", payload), height=580)

# ----- pressure graph
dmax = r["dB"] * 1.1 if r["dB"] > 0 else 1.0
d = np.linspace(0, dmax, 200)
dd = np.linspace(r["dT"], r["dB"], 60)
kpa = lambda depth: rho * g * depth / le3
fig, ax = plt.subplots(figsize=(8, 4.6))
ax.plot(kpa(d), d / uf, color="#7a8c9a", lw=1.4, label="Pressure in the liquid: p = ρ g d")
ax.fill_betweenx(dd / uf, 0, kpa(dd), color=col, alpha=0.6)
ax.plot(kpa(dd), dd / uf, color="#0b2a43", lw=3.5, label="Pressure on the plate")
ax.axhline(r["hc"] / uf, color="k", ls="--", lw=1.2, label=f"Centroid h̄ = {fnum(r['hc'] / uf)} {unit}")
ax.axhline(r["hcp"] / uf, color="#e0202a", ls="--", lw=1.2, label=f"Centre of pressure h_cp = {fnum(r['hcp'] / uf)} {unit}")
ax.plot([kpa(r["hc"])], [r["hc"] / uf], "ko", ms=8)
ax.plot([kpa(r["hcp"])], [r["hcp"] / uf], "o", color="#e0202a", ms=8)
ax.set_xlim(left=0)
ax.invert_yaxis()
ax.set_xlabel("Gauge pressure p (kPa)")
ax.set_ylabel(f"Depth below free surface ({unit})")
ax.set_title(f"Pressure distribution on the plate ({name})")
ax.grid(True, alpha=0.4)
ax.legend(loc="lower left", fontsize=8)
st.pyplot(fig)
plt.close(fig)

# ----- working + manual check
with st.expander("Step-by-step working (for report & viva)"):
    st.markdown(f"All lengths converted to metres · ρ = {fnum(rho)} kg/m³ · g = {fnum(g)} m/s² · θ = {theta}°")
    if shape == "Rectangular":
        st.latex(rf"A = b\cdot h = {fnum(b*uf, tex=True)}\times{fnum(h*uf, tex=True)} = {fnum(r['A'], tex=True)}\ \mathrm{{m^2}}")
        st.latex(rf"I_G = \frac{{b\cdot h^3}}{{12}} = {fnum(r['IG'], tex=True)}\ \mathrm{{m^4}}")
        st.latex(rf"\bar h = H + \frac{{h}}{{2}}\sin\theta = {fnum(r['hc'], tex=True)}\ \mathrm{{m}}")
    else:
        st.latex(rf"A = \pi R^2 = {fnum(r['A'], tex=True)}\ \mathrm{{m^2}}")
        st.latex(rf"I_G = \frac{{\pi R^4}}{{4}} = {fnum(r['IG'], tex=True)}\ \mathrm{{m^4}}")
        st.latex(rf"\bar h = H = {fnum(r['hc'], tex=True)}\ \mathrm{{m}}")
        st.latex(rf"F = \rho\cdot g\cdot\bar h\cdot A = {fnum(r['F'], tex=True)}\ \mathrm{{N}}")
        st.latex(rf"\bar y = \frac{{\bar h}}{{\sin\theta}} = {fnum(r['ybar'], tex=True)}\ \mathrm{{m}}")
        st.latex(rf"e = \frac{{I_G}}{{\bar y\cdot A}} = {fnum(r['e'], tex=True)}\ \mathrm{{m}}")
        st.latex(rf"h_{cp} = (\bar y + e)\sin\theta = \bar h + \frac{{I_G\sin^2\theta}}{{\bar h\cdot A}} = {fnum(r['hcp'], tex=True)}\ \mathrm{{m}}")

```

```

with st.expander("Manual (paper) vs app check"):
    st.write("Test problem: vertical rectangular gate 2 m wide × 3 m high in fresh water, top
    edge 1.5 m below the surface (g = 9.81 m/s²).")
    ex = compute("Rectangular", 2, 3, 90, 1.5, 1000, 9.81)
    st.table({
        "Quantity": ["Centroid depth  $\bar{h}$  (m)", "Total thrust F (kN)", "CP depth  $h_{cp}$  (m)"],
        "Manual (paper)": ["3.000", "176.58", "3.250"],
        "This app's formulas": [f"{ex['hc']:.3f}", f"{ex['F'] / 1e3:.2f}", f"{ex['hcp']:.3f}"],
    })
    st.caption("Paper:  $\bar{h} = 1.5 + 3/2 = 3 \text{ m}$  ·  $F = 1000 \times 9.81 \times 3 \times 6 = 176\,580 \text{ N}$  ·  $I_G = 2 \times 3^3 / 12 = 4.5 \text{ m}^4$  ·  $h_{cp} = 3 + 4.5 / (3 \times 6) = 3.25 \text{ m}$ ")

# ----- symbols reference
st.markdown("## Symbols & Meanings")
st.table({
    "Symbol": [
        "b", "h", "H", "A", "I_G", "θ", "ρ", "g",
        "h̄", "ȳ", "F", "e", "h_CP"
    ],
    "Meaning": [
        "Width of plate",
        "Height of plate",
        "Depth of upper edge",
        "Area of plate",
        "Moment of inertia about centroid",
        "Angle of inclination",
        "Fluid density",
        "Acceleration due to gravity",
        "Vertical depth of centroid",
        "Distance of centroid along plate",
        "Hydrostatic force",
        "Distance from centroid to centre of pressure",
        "Vertical depth of centre of pressure"
    ],
    "Unit": [
        "m", "m", "m", "m²", "m⁴", "degree", "kg/m³", "m/s²",
        "m", "m", "N", "m", "m"
    ]
})

st.markdown("---")
st.caption("Lalluvadiya Jenil Sagarbhai · Enrollment 25012251210001 · Diploma in Automation & Robotics Engineering · Lok Jagruti Kendra University")

```

6. SCREENSHOTS

The figures below are taken from the running Streamlit app (and from the same formulas plotted for the report). Figure 1 is the default rectangular-gate check case. Figure 2 is the deep circular hatch (submarine example). Figure 3 is the animated schematic at 100 m depth. Figure 4 is the step-by-step working printed by the app. Figure 5 is the pressure-distribution graph for the rectangular check case.

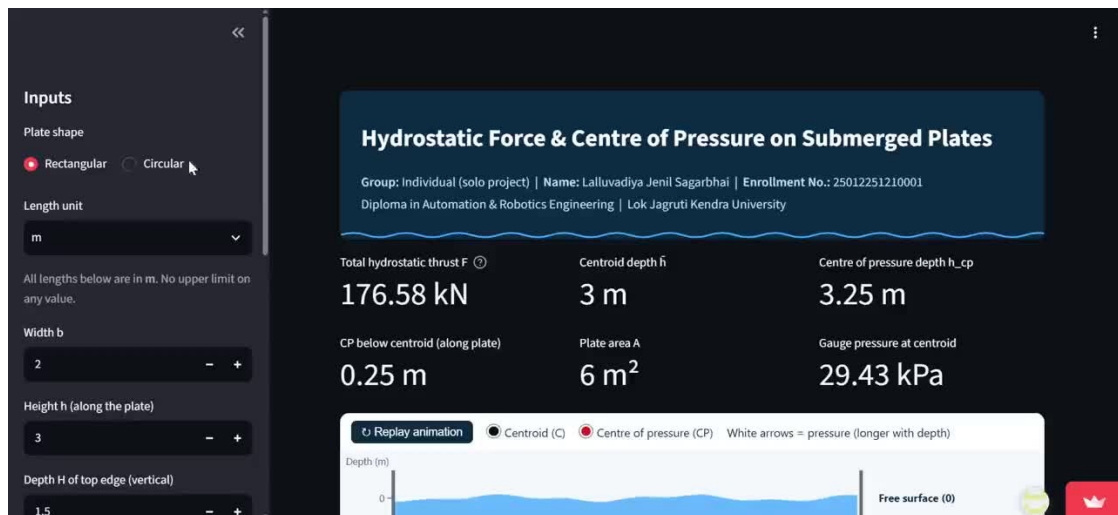


Figure 1. App output — rectangular gate 2 m × 3 m, top edge 1.5 m, fresh water ($F = 176.58$ kN, $h_{cp} = 3.25$ m).



Figure 2. App output — circular submarine hatch, $R = 0.5$ m, $H = 100$ m, sea water ($F = 789.74$ kN).

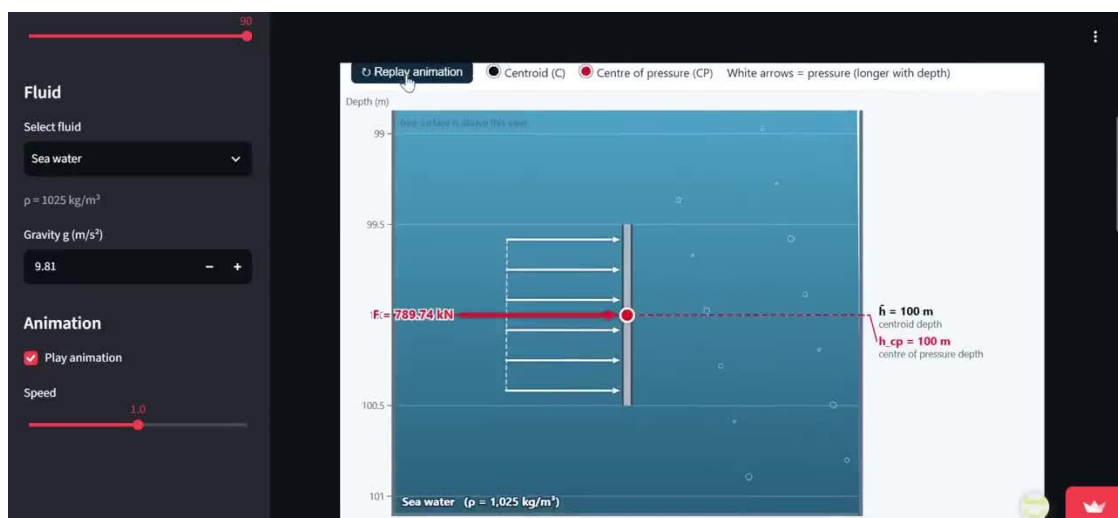


Figure 3. Animated schematic of the hatch: pressure arrows grow with depth; CP almost coincides with C.

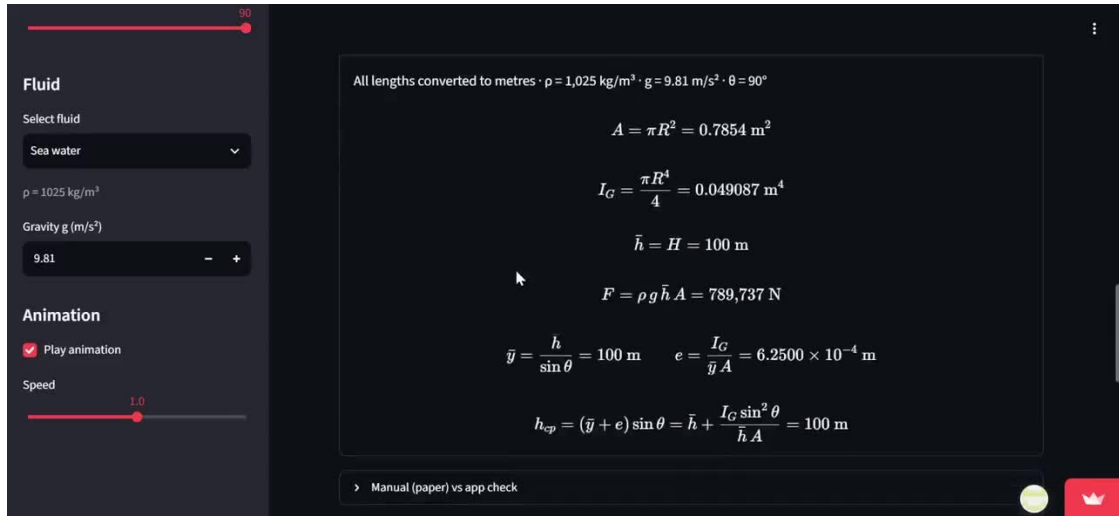


Figure 4. Step-by-step working printed by the app for the submarine-hatch inputs.

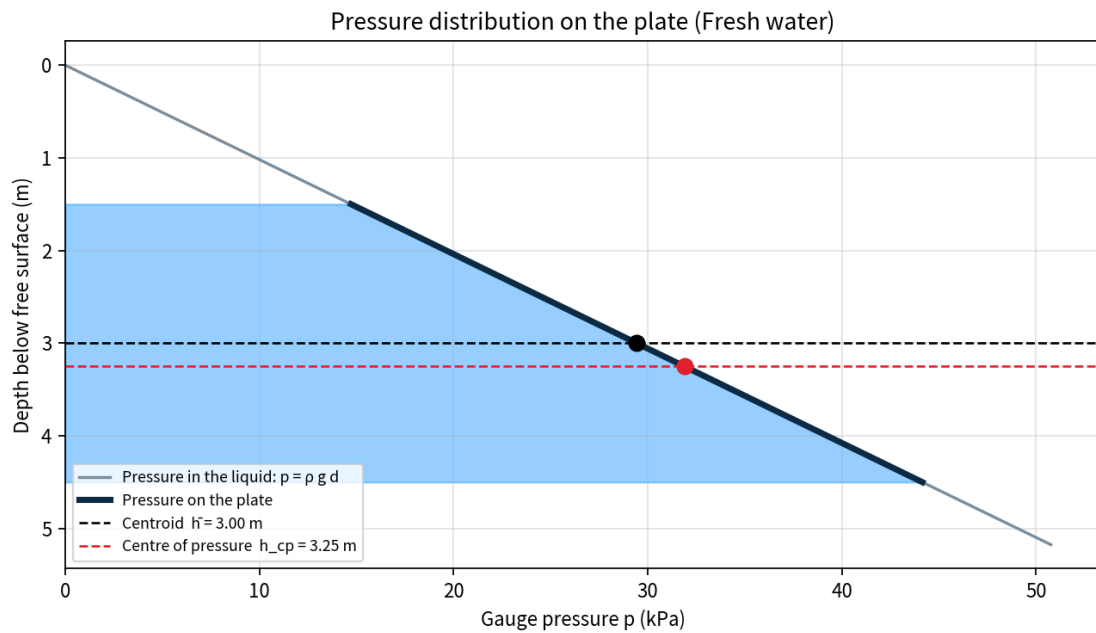


Figure 5. Pressure distribution on the rectangular plate (report plot from the same formulas).

7. RESULTS AND DISCUSSIONS

Two numerical cases were used to judge the app. Case A is a vertical rectangular gate, $b = 2 \text{ m}$, $h = 3 \text{ m}$, top edge $H = 1.5 \text{ m}$, fresh water, $g = 9.81 \text{ m/s}^2$. Paper working: $A = 6 \text{ m}^2$, $\bar{h} = 3.00$

m, $F = 1000 \times 9.81 \times 3 \times 6 = 176\,580 \text{ N} = 176.58 \text{ kN}$, $I_G = 4.5 \text{ m}^4$, $h_{cp} = 3 + 4.5/(3 \times 6) = 3.25 \text{ m}$. The app returns the same three values (0 % error). Case B is a vertical circular hatch, $R = 0.5 \text{ m}$, centre depth $H = 100 \text{ m}$, sea water $\rho = 1025 \text{ kg/m}^3$. The app gives $F = 789.74 \text{ kN}$, centroid pressure = 1 005.5 kPa and $e = 0.625 \text{ mm}$, so h_{cp} is 100.0006 m — the CP has almost merged with the centroid because the plate is tiny compared with its depth. That is the expected engineering behaviour: on a deep small lid the pressure is nearly uniform, so the resultant acts very near the centre. The pressure graph (Figure 5) is a straight line $p = \rho g d$; the portion that sits on the plate is shaded. The centroid and CP depths are marked and match the metrics. Tilting the plate (reducing θ) lowers both the vertical gap ($h_{cp} - \bar{h}$) and, for a rectangle whose top edge is fixed, the centroid depth itself.

Comparison table (Case A — rectangular gate in fresh water): Centroid depth $\bar{h} = 3.000 \text{ m}$ (paper) / 3.000 m (app); Total thrust $F = 176.58 \text{ kN}$ (paper) / 176.58 kN (app); CP depth $h_{cp} = 3.250 \text{ m}$ (paper) / 3.250 m (app). A second presentation slide used $H = 1.0 \text{ m}$ (instead of 1.5 m) and obtained $F = 147.15 \text{ kN}$ and $h_{cp} = 2.80 \text{ m}$; that pair also matches the same formulas with $\bar{h} = 2.5 \text{ m}$.

Formulas implemented: $F = \rho g \bar{h} A$; $h_{cp} = \bar{h} + I_G \sin^2\theta / (A \bar{h})$. Rectangle: $A = b h$, $I_G = b h^3 / 12$, $\bar{h} = H + (h/2) \sin\theta$. Circle: $A = \pi R^2$, $I_G = \pi R^4 / 4$, $\bar{h} = H$ (depth of centre).

8. CONCLUSION

The project objectives were met. A working Streamlit calculator now evaluates F and h_{cp} for rectangular and circular plates, at any inclination, in several fluids, with unit conversion, validation, animation, a pressure graph and a paper-style working. Manual and app results agree on the standard gate problem. Engineering conclusions from the runs are: thrust grows with centroid depth, area and density; the centre of pressure is always below the centroid on a fully submerged plate; the gap shrinks as the plate goes deeper; inclination cuts the vertical gap by $\sin^2\theta$; mercury gives about 13.6 times the thrust of fresh water. Limitations: the plate must be plane and fully submerged (the circle is not allowed to cut the free surface); only rectangle and circle are coded; the liquid is assumed still (no waves or flow); temperature effects on density are ignored except through the custom-fluid box. Possible extensions are triangular / elliptical plates, partially submerged gates, and a moment calculation about the hinges.

9. REFERENCES

1. Modi, P. N. and Seth, S. M., Hydraulics and Fluid Mechanics including Hydraulics Machines, Standard Book House. 2. Bansal, R. K., A Textbook of Fluid Mechanics and Hydraulic Machines, Laxmi Publications. 3. Streeter, V. L. and Wylie, E. B., Fluid Mechanics, McGraw-Hill. 4. Streamlit documentation, <https://docs.streamlit.io/> 5. NumPy and Matplotlib documentation. 6. Class notes, Semester-3 Python Programming and Fluid Mechanics, LJ Polytechnic, Lok Jagruti Kendra University.

10. COURSE CERTIFICATES

